



Download HTML



Download PDF

✈ NAVEEN AUTOMATION LABS ✈

Date: Oct 2, 2025

ELEMENT STATES 🎭

Visible, Enabled + Disabled

Before interacting with any element (clicking, typing, etc), we need to check its **state**! Is it visible? Is it enabled? Can we click on it? 🤔

Playwright gives us special methods to check element states and make our tests more reliable!

⚠ IMPORTANT!

Playwright automatically checks if element is visible, enabled, and stable before performing actions! But we can do manual checks too!

The 3 Element States 🎯

1 **isVisible()** 👁

- Checks if element is displayed on the page
- Returns: **true** or **false**

2 **isEnabled()** ✅

- Checks if element is enabled (can be interacted with)
- Returns: **true** or **false**

3 **isDisabled()** 🚫

- Checks if element is disabled (cannot be interacted with)
- Returns: `true` or `false`

 *Key Point:*

`isEnabled()` and `isDisabled()` are OPPOSITE of each other!

If `isEnabled() = true`, then `isDisabled() = false` 😊

Element States Visual

 *Button (Enabled)*

`isVisible() = true`

`isEnabled() = true`

`isDisabled() = false`



 *Button (Disabled)*

`isVisible() = true`

`isEnabled() = false`

`isDisabled() = true`

Real World Example

Let's check the Submit button state on a registration page - it's disabled until we accept terms!



Complete Example

```
await page.goto('https://classic.freecrm.com/register/');

let submitBtn: Locator = page.locator('#submitButton');

// Check initial state (BEFORE accepting terms)
let isVisible: boolean = await submitBtn.isVisible();
console.log(isVisible); // true

let isEnabled: boolean = await submitBtn.isEnabled();
console.log(isEnabled); // false ❌

let isDisabled: boolean = await submitBtn.isDisabled();
console.log(isDisabled); // true 🚫
```



Warning!

```
// ❌ DON'T DO THIS - Clicking disabled button will FAIL
await submitBtn.click(); // Error!
```

Playwright checks element is enabled before clicking - this will throw an error!



After Accepting Terms

```
// Accept terms and conditions
await page.locator('[name="agreeTerms"]').check();

// Check state again (AFTER accepting terms)
isVisible = await submitBtn.isVisible();
console.log(isVisible); // true

isEnabled = await submitBtn.isEnabled();
console.log(isEnabled); // true ✅

isDisabled = await submitBtn.isDisabled();
```

```
console.log(isDisabled); // false
```

```
// Now we can click safely! 🎉
```

```
await submitBtn.click();
```

State Change Flow

Before Checkbox is Checked:

Method	Returns
isVisible()	true
isEnabled()	false
isDisabled()	true



Checkbox Checked!



After Checkbox is Checked:

Method	Returns
isVisible()	true
isEnabled()	true 
isDisabled()	false

REMEMBER!

Playwright automatically waits for elements to be visible, enabled, and stable before actions like `click()`, `fill()`, etc. But manual checks are useful for validation!

Playwright's Auto-Waiting Feature

When you do `await element.click()`, Playwright automatically:

- ✅ Waits for element to be visible
- ✅ Waits for element to be enabled
- ✅ Waits for element to be stable (not moving)
- ✅ Only then performs the click!

🌟 This is why Playwright is smart!

You don't need explicit waits in most cases - Playwright handles it automatically!

When to Use Manual State Checks? 🤔

1 For Validation/Assertions

```
// Verify button is disabled
expect(await submitBtn.isDisabled()).toBe(true);
```

2 For Conditional Logic

```
// Check and act accordingly
if (await submitBtn.isEnabled()) {
  await submitBtn.click();
} else {
  console.log("Button is disabled!");
}
```

3 For Debugging

```
// Debug why element is not clickable
console.log('Visible:', await element.isVisible());
console.log('Enabled:', await element.isEnabled());
```

X COMMON MISTAKES

- × Trying to click on disabled elements
- × Confusing `isEnabled()` and `isDisabled()` (they're opposite!)
- × Not checking element state before assertions
- × Adding unnecessary waits (Playwright auto-waits!)
- × Forgetting that visible ≠ enabled (they're different!)

Quick Reference Card

Method	What it Checks	Return Type
<code>isVisible()</code>	Element displayed on page	boolean
<code>isEnabled()</code>	Element can be interacted	boolean
<code>isDisabled()</code>	Element cannot be interacted	boolean
<code>check()</code>	Checks a checkbox/radio	void

★ PRO TIPS!

- 💡 Combine checks with `expect()` for better assertions
- 💡 Trust Playwright's auto-waiting - don't add manual waits!

- 💡 Use state checks for validation, not for waiting
- 💡 Remember: `isEnabled()` and `isDisabled()` are opposites
- 💡 Element can be visible but disabled at the same time!

✨ KEY TAKEAWAYS

- Use `isVisible()`, `isEnabled()`, `isDisabled()` to check states
- Playwright auto-waits before actions - no manual waits needed!
- `isEnabled()` and `isDisabled()` are OPPOSITE
- Don't click disabled elements - Playwright will throw error
- Use state checks for validation and conditional logic 🎯

NAVEEN AUTOMATION LABS

Your Journey to Test Automation Excellence 🎯